รายงานโครงงาน ใบเสร็จดี

รายงานโครงงาน ใบเสร็จดี — Offline AI Receipt Analyzer

ระบบวิเคราะห์ใบเสร็จและสรุปค่าใช้จ่ายด้วยปัญญาประดิษฐ์แบบออฟไลน์บน Raspberry Pi

ผู้จัดทำ: **[กรอกชื่อ–นามสกุล]** รหัสนักศึกษา: **[กรอก]**

รายวิชา: **[กรอกชื่อรายวิชา]** อาจารย์ผู้สอน: **[กรอกชื่ออาจารย์]**

สาขาวิชา/สถานศึกษา: **[กรอก]** ภาคเรียน: **[กรอก]**

วันที่จัดทำเอกสาร: 25 ตุลาคม 2569

> เอกสารฉบับพร้อมตรวจแก้ก่อนส่ง: ผู้จัดทำต้องกรอกข้อมูลบนปก และเพิ่มภาพการทดสอบ Raspberry Pi จริงในหัวข้อ 9.3 ก่อนส่งวันที่ 31 ตุลาคม 2569

บทคัดย่อ

โครงงาน “ใบเสร็จดี” เป็นระบบเว็บไซต์สำหรับอ่านและบันทึกข้อมูลค่าใช้จ่ายจากภาพใบเสร็จ โดยประมวลผลบน Raspberry Pi 4 RAM 4 GB ภายในเครือข่ายท้องถิ่น ผู้ใช้ถ่ายภาพด้วยโทรศัพท์และอัปโหลดผ่านเว็บ ระบบปรับภาพและอ่านตัวอักษรภาษาไทยกับภาษาอังกฤษด้วย Tesseract OCR จากนั้นใช้กฎแยกบรรทัดร่วมกับโมเดลภาษา Qwen2.5-0.5B-Instruct ซึ่งทำงานผ่าน llama.cpp เพื่อสร้างร่างข้อมูล ได้แก่ ชื่อร้านหรือชื่อลูกค้า วันที่ รายการสินค้า/บริการ จำนวน และยอดเงิน ข้อมูลทุกช่องแสดงในหน้าตรวจแก้ก่อนบันทึกลง SQLite และสรุปผลบน Dashboard

จุดประสงค์สำคัญคือช่วยลดเวลาคัดลอกข้อมูล รักษาความเป็นส่วนตัวด้วยการไม่ส่งรูปหรือข้อความใบเสร็จไปยังบริการ AI ภายนอก และออกแบบให้ผู้ใช้เป็นผู้ยืนยันข้อมูลขั้นสุดท้าย ระบบรองรับ JPG/PNG ใบเสร็จพิมพ์ชัดเจนทั้งร้านค้าทั่วไปและใบรับบริการในขอบเขตต้นแบบ มีการตรวจชนิดไฟล์ ตรวจจำนวนเงินแบบทศนิยม ตรวจยอดสินค้า ส่วนลด และส่วนต่างก่อนบันทึก

ผลทดสอบอัตโนมัติล่าสุดจำนวน 50 กรณีผ่านทั้งหมด ครอบคลุม OCR การแยกใบเสร็จร้านค้าและใบรับบริการ การคำนวณส่วนลด การบันทึกแบบ transaction การป้องกันยืนยันซ้ำ และการแสดง Dashboard การติดตั้งบน Raspberry Pi สามารถเปิด OCR เซิร์ฟเวอร์โมเดล และเว็บไซต์ได้จริง รวมทั้งเข้าหน้าตรวจแก้จากภาพใบเสร็จได้ อย่างไรก็ตาม ความแม่นยำขึ้นกับคุณภาพภาพและรูปแบบใบเสร็จ จึงยังต้องให้ผู้ใช้ตรวจสอบ และต้องเก็บเวลา/ความถูกต้องจากใบเสร็จจริงหลายรูปแบบเพิ่มเติมก่อนสรุปประสิทธิภาพเชิงสถิติ

คำสำคัญ: Raspberry Pi, OCR, Local LLM, ใบเสร็จ, ระบบออฟไลน์, Flask, SQLite

1. บทนำ

1.1 ที่มาและความสำคัญของปัญหา

การบันทึกรายจ่ายจากใบเสร็จด้วยตนเองต้องอ่านข้อความ คัดลอกชื่อสินค้า ราคา วันที่ และจัดหมวดหมู่ซ้ำทุกครั้ง ขั้นตอนนี้ใช้เวลาและเกิดข้อผิดพลาดได้ง่าย โดยเฉพาะใบเสร็จที่มีหลายรายการหรือมีส่วนลด หากใช้บริการ AI บนอินเทอร์เน็ต ภาพใบเสร็จอาจมีชื่อ ที่อยู่ หมายเลขสมาชิก หรือข้อมูลการซื้อของผู้ใช้ที่ต้องส่งออกนอกอุปกรณ์

โครงงานนี้จึงพัฒนาระบบ Edge AI บน Raspberry Pi ให้โทรศัพท์ใน Wi-Fi เดียวกันใช้งานผ่านเว็บได้ โดย Raspberry Pi ทำหน้าที่เป็นทั้งเว็บเซิร์ฟเวอร์ เครื่องอ่าน OCR เครื่องรันโมเดลภาษา และฐานข้อมูล การออกแบบดังกล่าวช่วยลดการพึ่งพา API ภายนอก ไม่ต้องมีค่าเรียกใช้งานตามจำนวนครั้ง และยังใช้งานต่อได้เมื่อไม่มีอินเทอร์เน็ตขาออก ตราบใดที่เครือข่ายภายในยังทำงาน

เนื่องจาก OCR และโมเดลขนาดเล็กอาจอ่านผิด ระบบจึงใช้แนวคิด human-in-the-loop คือให้ AI สร้างร่างข้อมูล แต่ผู้ใช้ต้องเทียบกับภาพต้นฉบับ แก้ไข และกดยืนยันก่อนบันทึก วิธีนี้เหมาะกับต้นแบบที่ต้องการใช้งานจริงโดยไม่กล่าวอ้างว่า AI ถูกต้องทุกกรณี

1.2 ปัญหาที่ต้องการแก้ไข

1. การคัดลอกข้อมูลจากใบเสร็จด้วยมือใช้เวลานานและพิมพ์จำนวนเงินผิดได้
2. ใบเสร็จไทยมีรูปแบบ ตาราง คำย่อ และตำแหน่งยอดเงินแตกต่างกัน
3. ใบเสร็จบางใบมีส่วนลด ทำให้การรวมยอดจากรายการสินค้าอย่างเดียวคลาดเคลื่อน
4. การใช้ AI ภายนอกอาจต้องใช้อินเทอร์เน็ต มีค่าใช้จ่าย และส่งข้อมูลออกจากเครื่อง
5. ผล OCR/AI ต้องมีหน้าตรวจแก้เพื่อให้ผู้ใช้ควบคุมข้อมูลก่อนบันทึก

2. วัตถุประสงค์

1. พัฒนาเว็บที่รับภาพใบเสร็จ JPG/PNG จากโทรศัพท์หรือคอมพิวเตอร์ในเครือข่ายเดียวกับ Raspberry Pi
2. อ่านข้อความภาษาไทย ภาษาอังกฤษ และตัวเลขด้วย Tesseract OCR
3. ใช้กฎแยกข้อมูลและ Local LLM ช่วยกรอกชื่อร้าน วันที่ รายการ จำนวน และราคาโดยอัตโนมัติ
4. ให้ผู้ใช้แก้ไขข้อมูล พิมพ์หมวดหมู่เอง และตรวจสอบยอดรวม/ส่วนลดก่อนยืนยัน
5. บันทึกข้อมูลที่ยืนยันแล้วลง SQLite และแสดงรายจ่ายรวม กราฟหมวดหมู่ และประวัติบน Dashboard
6. ทดสอบการทำงานบน Raspberry Pi 4 RAM 4 GB และบันทึกข้อจำกัดอย่างตรงไปตรงมา

3. ขอบเขตของโครงงาน

3.1 ขอบเขตที่รองรับ

- ภาพ JPG หรือ PNG ขนาดไม่เกิน 10 MB และไม่เกิน 20 ล้านพิกเซล
- ใบเสร็จพิมพ์ชัดเจน ภาษาไทยและภาษาอังกฤษ 2–3 รูปแบบในระยะต้นแบบ
- ชื่อร้านหรือชื่อลูกค้า วันที่ รายการสินค้า/บริการ จำนวน ยอดรวมบรรทัด และยอดรวมใบเสร็จ
- ส่วนลดที่กรอกเป็นจำนวนบวก เช่น สินค้ารวม 104 บาท ลด 6 บาท ระบบคำนวณสุทธิ 98 บาท
- หมวดหมู่แบบข้อความอิสระ ผู้ใช้พิมพ์และแก้ไขเอง
- การเพิ่ม ลบ และแก้รายการก่อนยืนยัน
- Dashboard รายเดือน แสดงยอดรวม การแยกตามหมวดหมู่ และประวัติใบเสร็จ
- การใช้งานผ่าน localhost บน Pi หรือผ่าน IP ของ Pi จากเครื่องอื่นใน Wi-Fi เดียวกัน

3.2 สิ่งที่ยังไม่อยู่ในขอบเขต

- ใบเสร็จลายมือ PDF หลายหน้า หรือภาพที่ขาดรายการสำคัญ
- ระบบสมาชิก การแยกข้อมูลผู้ใช้หลายคน และการเปิดให้เข้าจากอินเทอร์เน็ตสาธารณะ
- การแก้หรือลบใบเสร็จหลังยืนยัน
- การรับประกันความถูกต้องกับใบเสร็จทุกรูปแบบ
- การฝึกโมเดลใหม่บน Raspberry Pi รุ่นนี้

4. หลักการและเทคโนโลยีที่เกี่ยวข้อง

4.1 Optical Character Recognition

OCR คือการแปลงข้อความในภาพเป็นตัวอักษร ระบบใช้ Tesseract พร้อมชุดภาษา `tha` และ `eng` ก่อนอ่านจะหมุนภาพตาม EXIF ย่อภาพขนาดใหญ่ ขยายภาพที่เล็ก เพิ่มความต่างระดับสี และเพิ่มความคม จากนั้นอ่านด้วย Page Segmentation Mode 6 และ 4 แล้วเลือกผลที่มีจำนวนบรรทัดเงินและคำสำคัญเหมาะสมกว่า วิธีนี้ช่วยรับมือทั้งใบเสร็จข้อความต่อเนื่องและใบเสร็จแบบตาราง

4.2 Local Large Language Model

ใช้ Qwen2.5-0.5B-Instruct รุ่น GGUF แบบ Q4_K_M ขนาดประมาณ 491 MB ผ่าน llama.cpp เนื่องจากเป็นโมเดลขนาดเล็กที่เหมาะกับหน่วยความจำจำกัด เว็บส่งเฉพาะข้อความ OCR ไปยังเซิร์ฟเวอร์โมเดลที่ `127.0.0.1:8081` และกำหนด JSON Schema เพื่อจำกัดรูปแบบคำตอบ โมเดลช่วยตีความข้อความที่จัดวางไม่เป็นระเบียบ แต่กฎ parser จะให้ความสำคัญกับราคาและจำนวนที่เห็นใน OCR เพื่อไม่ให้โมเดลสร้างตัวเลขขึ้นเอง

4.3 เว็บแอปพลิเคชันและฐานข้อมูล

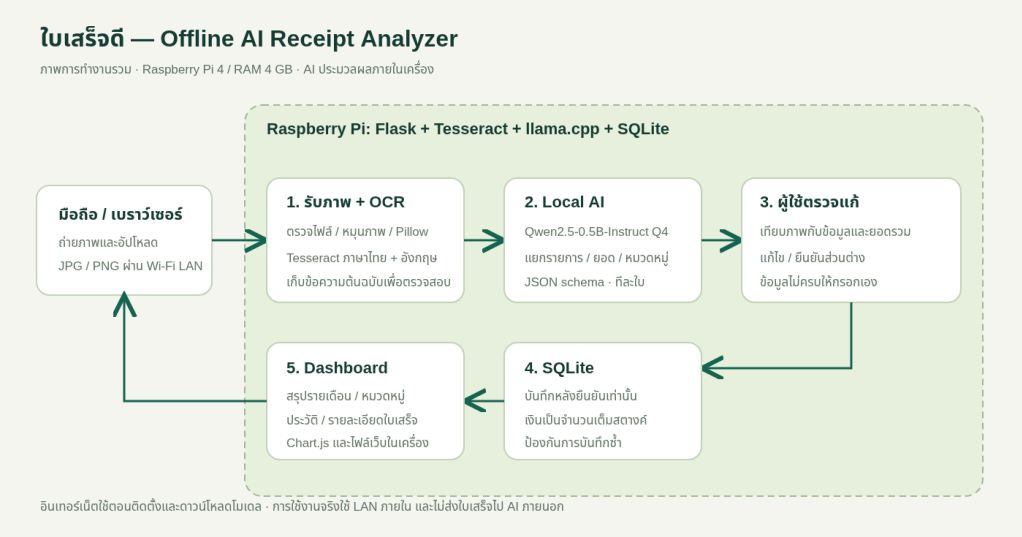
Flask รับคำขอจากเบราว์เซอร์และจัดคิววิเคราะห์ทีละงาน ส่วน Waitress ใช้ให้บริการบนพอร์ต 5000 ฐานข้อมูล SQLite เหมาะกับต้นแบบผู้ใช้จำนวนน้อยและเก็บข้อมูลในเครื่องเดียว Frontend ใช้ HTML, CSS และ JavaScript โดยเก็บ Bootstrap, Chart.js และฟอนต์ไว้ในโปรเจกต์ จึงไม่ต้องโหลด CDN ระหว่างใช้งาน

4.4 แนวคิด Human-in-the-loop

ระบบไม่บันทึกผล AI ทันที แต่แสดงภาพต้นฉบับคู่กับฟอร์ม ผู้ใช้ตรวจชื่อ วันที่ รายการ จำนวน และยอดเงินก่อนกดยืนยัน หาก AI ไม่พร้อม ระบบยังแสดงข้อความ OCR และให้กรอกเองได้ แนวคิดนี้ช่วยลดความเสี่ยงจากผลลัพธ์ที่คลาดเคลื่อนและทำให้ตรวจสอบที่มาของข้อมูลได้

5. การออกแบบระบบ

5.1 สถาปัตยกรรม



สถาปัตยกรรมและลำดับการทำงานของระบบ

โทรศัพท์หรือคอมพิวเตอร์เชื่อม Wi-Fi เดียวกับ Raspberry Pi แล้วเปิด `http://<IP ของ Pi>:5000` ภาพถูกส่งให้ Flask ตรวจชนิดและบันทึกในพื้นที่ instance จากนั้น worker เรียก Tesseract และ Local LLM ภายใน Pi ผลลัพธ์เข้าสู่หน้าตรวจแก้ เมื่อผู้ใช้ยืนยันจึงบันทึกลง SQLite และนำไปคำนวณ Dashboard

5.2 ลำดับการทำงาน

1. ผู้ใช้เลือกภาพและกดเริ่มอ่านใบเสร็จ
2. ระบบตรวจนามสกุล ชนิดภาพ ขนาดไฟล์ และจำนวนพิกเซล
3. OCR ปรับภาพและอ่านข้อความสองรูปแบบการจัดหน้า
4. parser แยกวันที่ ลูกค้า/ร้าน และบรรทัดที่ลงท้ายด้วยจำนวนเงิน
5. Local LLM ช่วยเติมข้อมูลที่ parser ยังหาไม่ได้ โดยคืน JSON ตาม schema
6. ระบบรวมผลโดยยึดจำนวนเงินจาก OCR เป็นหลัก และเว้นหมวดหมู่ให้ผู้ใช้พิมพ์
7. ผู้ใช้ตรวจแก้รายการ กรอกส่วนลด และตรวจยอดสุทธิ
8. ระบบตรวจข้อมูลอีกครั้งและบันทึกใบเสร็จกับรายการใน transaction เดียว
9. Dashboard อ่านเฉพาะใบเสร็จสถานะ confirmed

5.3 สถานะของใบเสร็จ

สถานะทำงานคือ `queued → processing → review → confirmed` หาก OCR หรือ AI เกิดข้อผิดพลาด ระบบเปลี่ยนเข้าสู่ review พร้อมคำอธิบายและเปิดให้กรอกเอง หากโปรแกรมถูกปิดระหว่างวิเคราะห์ เมื่อเปิดใหม่งานค้างจะกลับมาให้ตรวจแทนการแสดงว่าวิเคราะห์สำเร็จ

5.4 โครงสร้างข้อมูล

ตาราง	ข้อมูลสำคัญ	หน้าที่
receipts	id, image, OCR text, status, merchant, date, total, discount, timestamps	เก็บข้อมูลระดับใบเสร็จและสถานะการวิเคราะห์
receipt_items	receipt_id, name, quantity, line_total, category	เก็บสินค้า/บริการแต่ละบรรทัด

ยอดเงินเก็บเป็นจำนวนเต็มสตางค์แทนเลขทศนิยมแบบ floating point เพื่อหลีกเลี่ยงความคลาดเคลื่อน เช่น 0.1 + 0.2 การยืนยันใช้ transaction และป้องกันการกดซ้ำ เพื่อไม่ให้ใบเสร็จเดียวถูกบันทึกสองครั้ง

6. อุปกรณ์และซอฟต์แวร์

ประเภท	รายการ	หน้าที่
Hardware	Raspberry Pi 4 RAM 4 GB	ประมวลผล OCR, AI, เว็บ และฐานข้อมูล
Hardware	microSD card และแหล่งจ่ายไฟ	เก็บระบบปฏิบัติการ/ข้อมูล และจ่ายไฟให้ Pi
Hardware	พัดลมหรืออีตซิงก์	ลดความร้อนระหว่างรันโมเดล
Client	โทรศัพท์หรือคอมพิวเตอร์	ถ่าย/เลือกภาพ ตรวจแก้ และดู Dashboard
Network	Wi-Fi/เราเตอร์ภายใน	เชื่อมผู้ใช้กับเว็บบน Pi
OS	Raspberry Pi OS 64-bit	ระบบปฏิบัติการ aarch64
Backend	Python 3.11+, Flask, Waitress	เว็บและกระบวนการวิเคราะห์
OCR	Tesseract OCR, Pillow	ปรับภาพและอ่านข้อความไทย/อังกฤษ
AI	Qwen2.5-0.5B-Instruct GGUF, llama.cpp	วิเคราะห์ข้อความในเครื่อง
Database	SQLite	เก็บใบเสร็จและรายการ
Frontend	HTML, CSS, JavaScript, Bootstrap, Chart.js	หน้าจอและกราฟ

6.1 งบประมาณ

รายการ	จำนวน	ราคา (บาท)	หมายเหตุ
Raspberry Pi 4 RAM 4 GB	1	[กรอก]	[ซื้อใหม่/อุปกรณ์ที่มีอยู่]
microSD card	1	[กรอก]	ความจุ [กรอก] GB
แหล่งจ่ายไฟ/เคส/ระบายความร้อน	1 ชุด	[กรอก]	[กรอก]
ซอฟต์แวร์โอเพนซอร์ส	-	0	ไม่มีค่า API ระหว่างใช้งาน
รวม		**[กรอกยอดจริง]**	ใช้ราคาจากหลักฐานจริง

7. แผนการพัฒนาและการดำเนินงาน

วัน	งาน	ผลลัพธ์
Day 1	เตรียม Pi และทดสอบ OCR	Tesseract ภาษาไทย/อังกฤษพร้อมใช้
Day 2	ติดตั้ง llama.cpp และ Qwen	เซิร์ฟเวอร์โมเดลตอบ health check
Day 3	พัฒนาหน้าเว็บและอัปโหลด	รับภาพจากโทรศัพท์ได้
Day 4	เชื่อม OCR, parser และ Local LLM	สร้างร่างข้อมูลอัตโนมัติและตรวจแก้ได้
Day 5	พัฒนาฐานข้อมูลและ Dashboard	บันทึกและสรุปรายจ่ายได้
Day 6	รวมระบบและทดสอบบน Pi	พบข้อผิดพลาดจากใบเสร็จจริงและปรับกฎ
Day 7	แก้ไข ทดสอบซ้ำ และเตรียมนำเสนอ	รายงาน คู่มือ และบทคลิป

การพัฒนาจริงใช้การทดสอบวนซ้ำ ตัวอย่างเช่น ใบรับบริการแบบตารางเคยนำชื่อลูกค้าไปเป็นชื่อสินค้าและอ่านจำนวนผิด จึงเพิ่มกฎแยกคอลัมน์รายละเอียด จำนวน ราคาต่อหน่วย และยอดบรรทัด อีกกรณีคือส่วนลด 6 บาทจากยอดสินค้า 104 บาทเคยถูกรวมผิด จึงเปลี่ยนช่องให้รับ “ส่วนลดจำนวนบวก” และคำนวณเป็น 104 − 6 = 98 บาท พร้อมเพิ่ม automated test ป้องกันการเกิดซ้ำ

8. การพัฒนาโปรแกรมและโค้ดสำคัญ

8.1 โครงสร้างโปรแกรม

ไฟล์/โฟลเดอร์	หน้าที่
app.py	route, session, คิววิเคราะห์ และการเชื่อมแต่ละ service
services/ocr.py	ปรับภาพและเรียก Tesseract
services/receipt_parser.py	แยกบรรทัดสินค้า ตารางบริการ วันที่ และยอดรวม
services/llm.py	ติดต่อ llama.cpp เฉพาะ loopback และกำหนด JSON Schema
services/validation.py	ตรวจราคา จำนวน วันที่ ส่วนลด และส่วนต่าง
services/database.py	schema, transaction และ query Dashboard
templates/, static/	หน้าเว็บ สไตล์ JavaScript และไฟล์ vendor ในเครื่อง
scripts/	ติดตั้ง เริ่มบริการ ตรวจสุขภาพ และทดสอบ
tests/	automated tests ของ parser และ workflow

8.2 ตัวอย่างการอ่าน OCR สองรูปแบบ

```
candidates = [
    pytesseract.image_to_string(
        image, lang="tha+eng", config=f"--psm {psm}", timeout=90
    ).strip()
    for psm in (6, 4)
]
return max(candidates, key=score)
```

ระบบเลือกข้อความที่มีบรรทัดจำนวนเงินและคำสำคัญ เช่น total, รวม, จำนวน หรือ ลูกค้า มากกว่า แทนการเชื่อผลจากการตั้งค่าเดียว

8.3 ตัวอย่างการตรวจจำนวนเงิน

```
amount = Decimal(str(value).strip())
if amount != amount.quantize(Decimal("0.01")):
    raise ValueError("จำนวนเงินต้องมีไม่เกิน 2 ตำแหน่ง")
amount_satang = int(amount * 100)
```

ยอดรวมที่ระบบคำนวณคือผลรวมยอดบรรทัดสินค้า ลบส่วนลด และรวมส่วนต่างที่ผู้ใช้ยืนยัน ระบบไม่คูณยอดบรรทัดด้วยจำนวนซ้ำ เพราะ `line_total` หมายถึงยอดรวมของบรรทัดนั้นแล้ว

8.4 การจำกัด AI ให้อยู่ในเครื่อง

`LLAMA_URL` ยอมรับเฉพาะ `127.0.0.1`, `localhost` หรือ `::1` ผ่าน HTTP และปิดการใช้ proxy ของ requests เว็บจึงไม่สามารถตั้งค่าให้ส่งข้อความใบเสร็จไปโดเมนภายนอกโดยไม่แก้โค้ด นอกจากนี้ llama.cpp ฟังเฉพาะ loopback ส่วนเว็บฟังพอร์ต 5000 ภายใน LAN

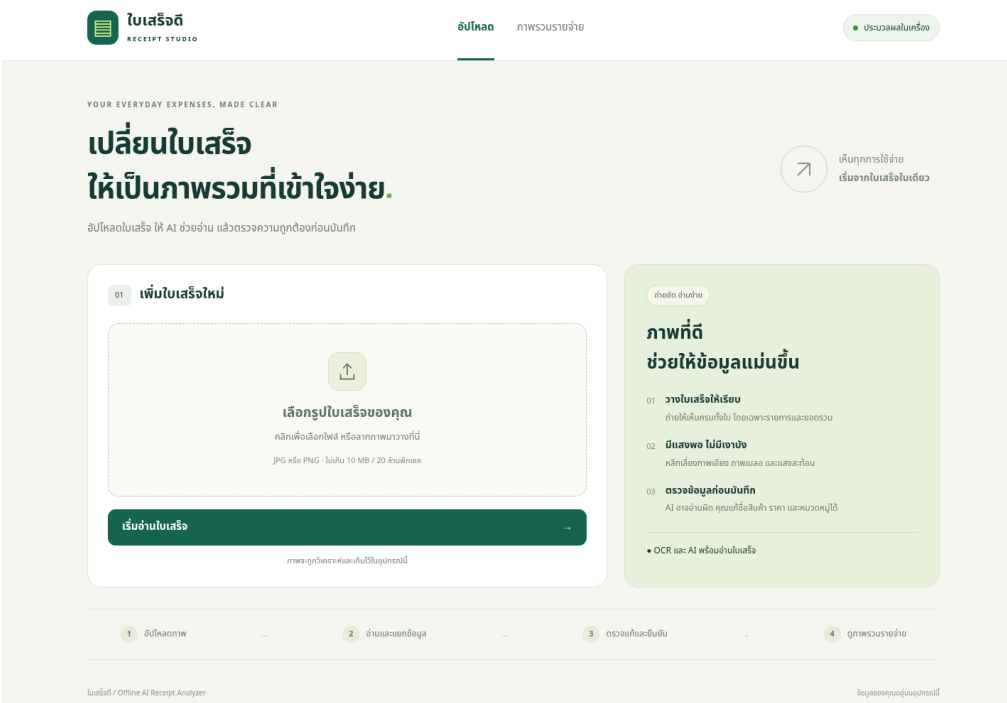
9. การทดสอบและผลการทดลอง

9.1 วิธีทดสอบ

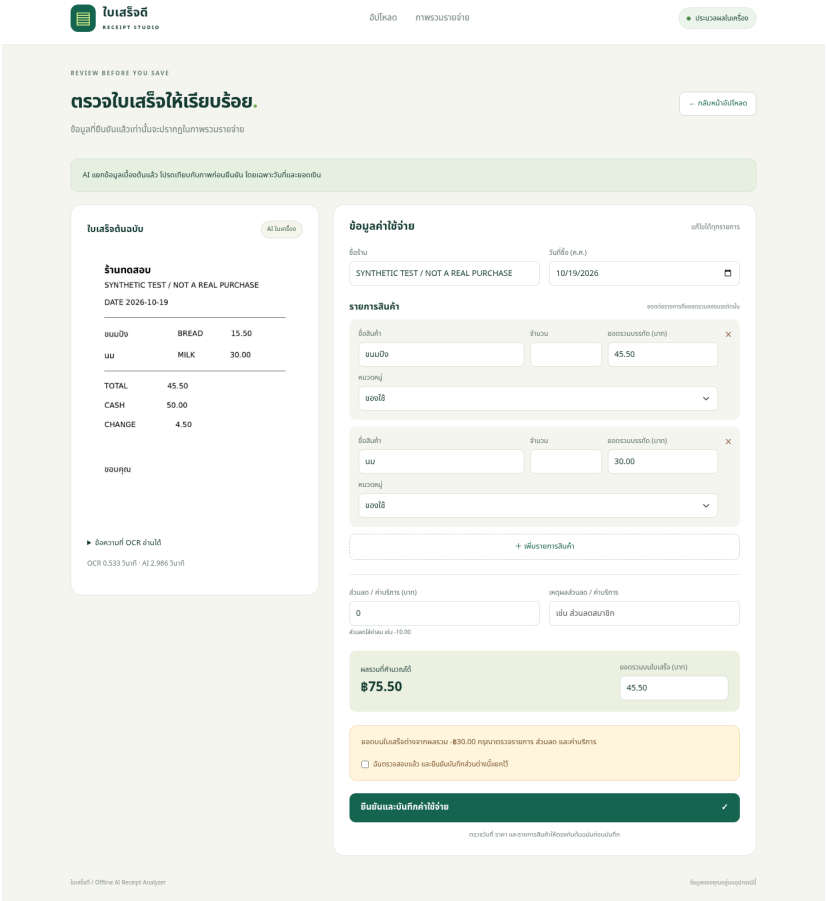
การทดสอบแบ่งเป็นสามระดับ ได้แก่ automated test ของฟังก์ชันย่อยและ workflow, การทดสอบผ่าน Chromium ด้วยภาพสังเคราะห์ที่ทราบคำตอบ และการทดลองบน Raspberry Pi ด้วยภาพใบเสร็จจริง ผลจากภาพสังเคราะห์ใช้ตรวจว่าท่อการทำงานครบ ไม่ใช้กล่าวอ้างความแม่นยำของใบเสร็จทุกชนิด

9.2 ผลทดสอบในสภาพแวดล้อมพัฒนา

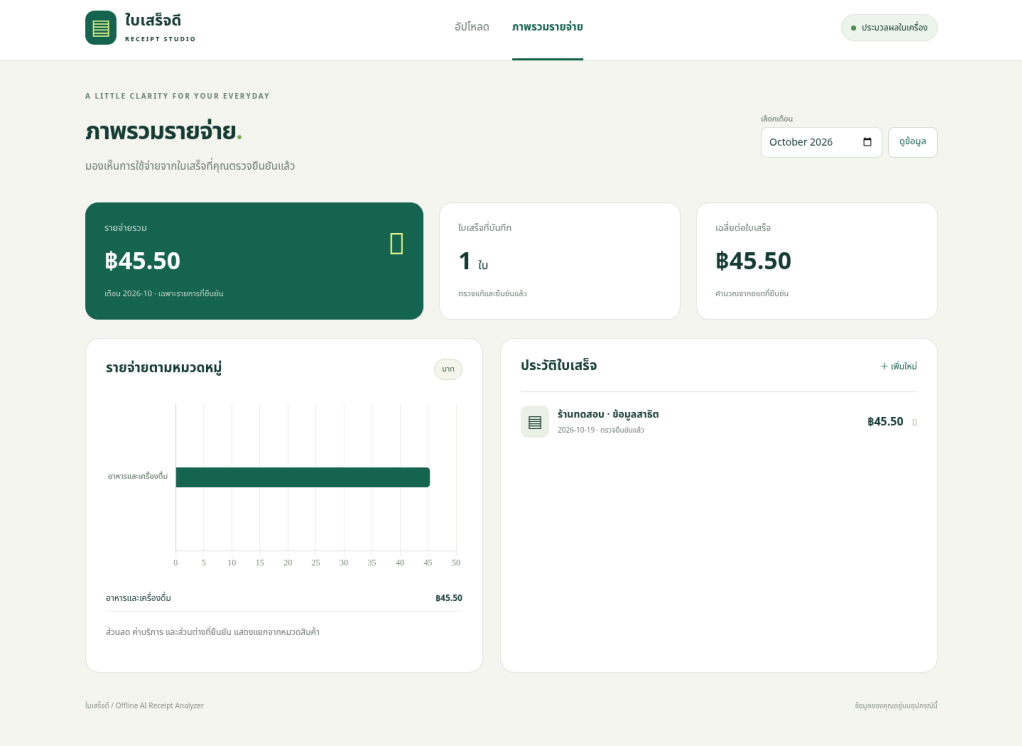
การทดสอบ	ผลล่าสุด	ขอบเขตของหลักฐาน
Pytest	**ผ่าน 50/50, ไม่ผ่าน 0**	XML วันที่ 24 ต.ค. 2569; parser, เงิน, ส่วนลด, transaction, security, upload และ workflow
OCR ไทย/อังกฤษ	ผ่าน	ภาพสังเคราะห์ อ่านคำไทย อังกฤษ และราคาได้
Browser workflow	ผ่าน	อัปโหลด → OCR → Local AI → ตรวจแก้ → บันทึก → Dashboard
Responsive 390 px	ผ่าน	ไม่มี horizontal overflow ในหน้าหลัก
โหลดหน้าเว็บโดยบล็อกอินเทอร์เน็ตภายนอก	ผ่าน	ไม่มี external browser request หรือ JavaScript error
llama.cpp + Qwen จริงบน x86_64	เรียกใช้งานสำเร็จ	OCR ประมาณ 0.41 วินาที, AI ประมาณ 4.59 วินาที ในตัวอย่างหนึ่ง; ไม่ใช่ความเร็วบน Pi
ความถูกต้อง AI จากตัวอย่างเดียว	ยังสรุปไม่ได้	โมเดลคืน JSON ได้แต่เคยอ่านชื่อ/ราคา/หมวดหมู่ผิด จึงต้องตรวจแก้



หน้าอัปโหลดภาพใบเสร็จ — ข้อมูลสาธิต



ข้อมูลที่ระบบกรอกอัตโนมัติก่อนผู้ใช้ตรวจแก้ — ข้อมูลสาธิต



Dashboard สรุปรายจ่ายและประวัติ — ข้อมูลสาธิต

9.3 ผลที่ตรวจบน Raspberry Pi จริง

การติดตั้งบน Raspberry Pi 4 RAM 4 GB ผ่านขั้นตอน build llama.cpp ดาวน์โหลดโมเดล เริ่ม Local LLM และเริ่มเว็บ ค่า readiness ของเว็บ ฐานข้อมูล OCR และ AI แสดง `true` ทั้งหมด ผู้ใช้เปิดเว็บที่ `127.0.0.1:5000` บน Pi และอัปโหลดใบเสร็จจนระบบแสดงร่างข้อมูลอัตโนมัติได้จริง การทดลองทำให้พบกรณีใบรับบริการแบบตารางและส่วนลดของใบเสร็จร้านสะดวกซื้อ ซึ่งนำไปสู่การแก้ parser และสูตรส่วนลดตามหัวข้อ 7

ยังไม่มีการบันทึกเวลาและ ground truth ครบ 10 ใบ จึงไม่คำนวณเปอร์เซ็นต์ความแม่นยำในรายงานฉบับนี้ ตารางต่อไปนี้เตรียมไว้ให้ผู้จัดทำทดสอบเวอร์ชันล่าสุดก่อนส่ง

ลำดับ	รูปแบบใบเสร็จ	เวลา OCR+AI	ร้าน/ลูกค้า	รายการ	จำนวน	ยอดสุทธิ	จำนวนช่องที่แก้
1	ใบรับบริการล้างแอร์	[กรอก]	[ผ่าน/ไม่ผ่าน]	[กรอก]	[กรอก]	1,600.00	[กรอก]
2	ร้านสะดวกซื้อ มีส่วนลด	[กรอก]	[กรอก]	[กรอก]	[กรอก]	98.00	[กรอก]
3–10	ใบเสร็จจริงเพิ่มเติม	[กรอก]	[กรอก]	[กรอก]	[กรอก]	[กรอก]	[กรอก]

ภาพที่ต้องเพิ่มก่อนส่ง: ภาพ Pi และหน้าจอเว็บขณะเปิดระบบ, หน้าตรวจแก้ใบรับบริการหลังอัปเดต, หน้าใบเสร็จร้านสะดวกซื้อที่ยอดสุทธิ 98 บาท และ Dashboard จากข้อมูลทดสอบจริง

9.4 การวิเคราะห์ผล

ระบบทำ workflow หลักได้ครบและการคำนวณเงินมี automated test รองรับ จุดอ่อนหลักอยู่ที่ข้อมูลนำเข้า เพราะ OCR อาจทำให้ตัวอักษรไทยติดกันและลำดับคอลัมน์เปลี่ยน ส่วนโมเดล 0.5B มีความสามารถจำกัด การใช้ parser ร่วมกับ LLM จึงให้ผลควบคุมง่ายกว่าการให้โมเดลเดาตัวเลขทั้งหมด แต่ยังต้องปรับกฎจากตัวอย่างใบเสร็จเป้าหมาย 2–3 รูปแบบ

การมีหน้าตรวจแก้ช่วยให้ระบบยังใช้งานได้แม้ร่างอัตโนมัติผิดบางช่อง ผู้ใช้ไม่จำเป็นต้องฝึกโมเดลใหม่สำหรับทุกข้อผิดพลาด หากรูปแบบใบเสร็จมีโครงสร้างคงที่ สามารถเพิ่ม regex, คำสำคัญ หรือตัวอย่าง prompt ได้ การ fine-tune ควรพิจารณาเมื่อมีชุดข้อมูลที่ปกปิดข้อมูลส่วนบุคคลและติดป้ายกำกับจำนวนมากเพียงพอ

10. จุดเด่นและนวัตกรรม

นวัตกรรมของโครงงานอยู่ที่การประยุกต์ OCR ภาษาไทย Local LLM และระบบบัญชีรายจ่ายให้ทำงานร่วมกันบนคอมพิวเตอร์ขนาดเล็ก โดยออกแบบการตรวจสอบยอดและการแก้ข้อมูลเป็นส่วนหนึ่งของ workflow ไม่ใช่เพียงสาธิตการอ่านข้อความ ระบบยังทำงานผ่านเว็บที่โทรศัพท์เข้าถึงได้ จึงใช้ Raspberry Pi เป็น Edge server ภายในเครือข่าย

ประเด็น	กรอกข้อมูลด้วยมือ	ระบบสแกนแบบ Cloud ทั่วไป	ใบเสร็จดี
อ่านข้อมูลอัตโนมัติ	ไม่มี	มี	OCR + parser + Local LLM
ต้องส่งข้อมูลออกอินเทอร์เน็ต	ไม่ต้อง	ขึ้นกับบริการ	ไม่ต้องระหว่างใช้งาน
ทำงานบนอุปกรณ์ผู้ใช้	ได้	ส่วนประมวลผลหลักอยู่นอกเครื่อง	ทำงานบน Raspberry Pi
ตรวจแก้ก่อนบันทึก	ทำเองทั้งหมด	ขึ้นกับบริการ	มีหน้าตรวจภาพและแก้ทุกช่อง
ตรวจยอด/ส่วนลด	ทำเอง	ขึ้นกับบริการ	ตรวจสูตรและแจ้งส่วนต่าง
ค่า API ต่อครั้ง	ไม่มี	อาจมี	ไม่มี
ข้อจำกัด	ช้าและผิดจากการพิมพ์	ต้องพึ่งผู้ให้บริการ	ช้ากว่า Cloud และแม่นยำตามคุณภาพภาพ

โครงงานนี้เป็นการต่อยอดเทคโนโลยีที่มีอยู่ด้วยการออกแบบระบบเฉพาะโจทย์ จึงจัดเป็นนวัตกรรมเชิงประยุกต์ ไม่กล่าวอ้างว่าเป็นแนวคิดที่ไม่เคยมีในตลาด

11. ความปลอดภัย ความเป็นส่วนตัว และความน่าเชื่อถือ

1. เก็บรูป ฐานข้อมูล และ session key ในโฟลเดอร์ `instance/` บน Pi และไม่บันทึกลง Git
2. ยอมรับเฉพาะไฟล์ภาพที่ตรวจเปิดได้ จำกัดขนาด และ normalize ก่อนใช้งาน
3. ใช้ CSRF token กับฟอร์มและ escape ข้อความก่อนแสดง เพื่อลดการโจมตีผ่านเว็บ
4. AI รับคำขอผ่าน loopback เท่านั้น และ frontend ไม่เรียก CDN
5. ใช้ใน LAN ที่ไว้ใจได้ เพราะรุ่นนี้ยังไม่มีบัญชีผู้ใช้
6. ควรหยุดบริการเว็บก่อนสำรองโฟลเดอร์ `instance/` และไม่เปิด port forwarding จากอินเทอร์เน็ต

12. ปัญหา ข้อจำกัด และแนวทางต่อยอด

12.1 ปัญหาที่พบ

- การย้ายโฟลเดอร์โปรเจกต์ทำให้ CMake cache ชี้พาธเก่า ต้องสร้าง build directory ใหม่
- ใบรับบริการแบบตารางทำให้ OCR วางชื่อลูกค้าและคอลัมน์สินค้าในลำดับที่ parser รุ่นแรกตีความผิด
- ใบเสร็จร้านสะดวกซื้อมีโปรโมชั่น คำย่อ และส่วนลดหลายบรรทัด
- Local LLM ขนาดเล็กใช้เวลามากกว่าคอมพิวเตอร์ทั่วไปและอาจคืนข้อมูลไม่ครบ

12.2 แนวทางต่อยอด

- เก็บชุดทดสอบใบเสร็จจริงอย่างน้อย 10–30 ใบต่อรูปแบบ โดยปกปิดข้อมูลส่วนบุคคล
- เพิ่ม template detector สำหรับใบเสร็จร้านเป้าหมาย เช่น ร้านสะดวกซื้อและใบรับบริการ
- เพิ่มการ crop/deskew และการตรวจตำแหน่งตารางด้วย OpenCV
- เพิ่มหน้าสำหรับแก้หรือลบรายการหลังยืนยันและระบบสำรองข้อมูลอัตโนมัติ
- เพิ่มบัญชีผู้ใช้และ HTTPS หากต้องใช้นอก LAN ที่ควบคุม
- ทดลองโมเดลหรือ quantization อื่น แล้วเปรียบเทียบความเร็ว หน่วยความจำ และความถูกต้องด้วยชุดข้อมูลเดียวกัน

13. สรุป

โครงงานใบเสร็จดีพัฒนาระบบตั้งแต่รับภาพ อ่าน OCR วิเคราะห์ด้วย Local AI ตรวจแก้ บันทึกฐานข้อมูล และแสดง Dashboard ได้ครบในเว็บเดียว ระบบทำงานบน Raspberry Pi และเปิดให้เครื่องอื่นใน Wi-Fi เดียวกันใช้ผ่าน IP ของ Pi ได้ การประมวลผลหลักอยู่ในอุปกรณ์ ช่วยรักษาความเป็นส่วนตัวและลดการพึ่งพา API ภายนอก

Automated test 50 กรณีผ่านทั้งหมดและการทดลองบน Pi ยืนยันว่าองค์ประกอบหลักเริ่มทำงานร่วมกันได้ การทดสอบกับใบเสร็จจริงทำให้พบและแก้ปัญหาการแยกตารางบริการกับการคำนวณส่วนลด อย่างไรก็ตาม OCR และโมเดลขนาดเล็กยังไม่สามารถรับประกันความถูกต้องทุกใบ ระบบจึงให้ผู้ใช้ตรวจข้อมูลก่อนบันทึก การเก็บผลเวลาและความถูกต้องจากใบเสร็จจริงหลายใบเป็นงานสำคัญที่ต้องทำก่อนนำไปใช้ในวงกว้าง

14. เอกสารอ้างอิง

1. Tesseract OCR Documentation. https://tesseract-ocr.github.io/
2. Flask Documentation. https://flask.palletsprojects.com/
3. SQLite Documentation. https://www.sqlite.org/docs.html
4. ggml-org. llama.cpp. https://github.com/ggml-org/llama.cpp
5. Qwen Team. Qwen2.5-0.5B-Instruct-GGUF. https://huggingface.co/Qwen/Qwen2.5-0.5B-Instruct-GGUF
6. Chart.js Documentation. https://www.chartjs.org/docs/
7. Bootstrap Documentation. https://getbootstrap.com/docs/
8. เกณฑ์การประเมิน Project IoT (20 คะแนน) เอกสารที่อาจารย์ผู้สอนกำหนด

ภาคผนวก ก คำสั่งเริ่มระบบบน Raspberry Pi

```
cd ~/Downloads/ProjectIOT
bash scripts/start_llm.sh
```

เปิด terminal ที่สอง:

```
cd ~/Downloads/ProjectIOT
bash scripts/start_web.sh
```

เปิดเว็บบน Pi ที่ `http://127.0.0.1:5000` หรือดู IP ด้วย `hostname -I` แล้วเปิดจากเครื่องอื่นเป็น `http://<IP>:5000` ทั้งสองเครื่องต้องอยู่ Wi-Fi เดียวกัน

ภาคผนวก ข รายการหลักฐานก่อนส่ง

- [ ] กรอกชื่อผู้จัดทำ รหัส รายวิชา อาจารย์ สาขา และงบประมาณจริง
- [ ] ทดสอบเวอร์ชันล่าสุดบน Pi กับใบรับบริการและใบเสร็จร้านสะดวกซื้อ
- [ ] เติมตารางผลทดสอบ 10 ใบและเวลาที่วัดจริง
- [ ] เพิ่มภาพ Pi จริง หน้าตรวจแก้จริง และ Dashboard จริง
- [ ] เปิดตรวจการจัดหน้าและเลขหน้าใน Microsoft Word/LibreOffice
- [ ] ตรวจคำผิดและส่งทั้ง DOCX กับ PDF ภายใน 31 ตุลาคม 2569